

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 3: Neural Networks: Feedforward Networks and the Training Process

Student Name: Aluong Leek Alier

Leek _____

Student ID: _78152028_____

GitHub Repository Link:

___https://github.com/aluongleekalierleek/Lab_3.git_____

Date Submitted: ___26/07/2026_____

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	yes
Estimated percentage of the work influenced by AI	20 %
Did you attach evidence of AI use where applicable?	yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
gemini	To learn concepts			
	To learn gradient descent math	Show me how the weight update formula works for a single neuron with input x , weight w , and output y	Helped me understand the update rule and apply it in my notebook	I checked the formulas, applied them in code, and explained them in simple words
	To learn differences in learning rates	What differences should I expect in the loss curve when using learning rates 0.01, 0.1, and 1.0?	Helped me compare slow, stable, and unstable training behaviors	I plotted the loss curves, confirmed the differences, and added my own notes

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name	
AI-generated draft	yes		
Prompt history	yes		
Screenshot of AI interaction	yes		Show me how the weight update formula works for a single neuron with input x, weight w, and output y Here's the step-by-step explanation of the weight update formula for a single neuron with input x, weight w, and output y:  Single Neuron Setup • Input: x • Weight: w • Prediction: $\hat{y} = w \cdot x$ • True output: y • Loss function:

- Loss function:

$$L = (y - \hat{y})^2$$

Gradient of the Loss

To update the weight, we need the derivative of the loss with respect to w :

$$\frac{dL}{dw} = -2x(y - \hat{y})$$

This tells us how much the loss changes when w changes.

Update Rule

Gradient descent update:



Gradient descent update:

$$w_{\text{new}} = w_{\text{old}} - \text{lr} \cdot \frac{dL}{dw}$$

- **lr** = learning rate (controls step size).
- If lr is small $\rightarrow$ slow but stable updates.
- If lr is large $\rightarrow$ fast but unstable updates.

✓ Example

Suppose:

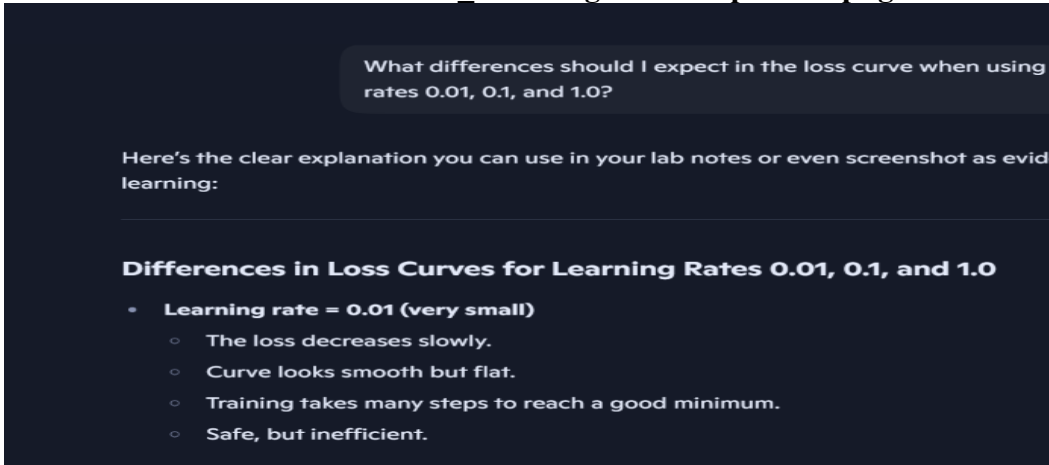
- $x = 2, y = 10, w = 1$, learning rate = 0.1.
- Prediction: $\hat{y} = 1 \cdot 2 = 2$.
- Loss: $L = (10 - 2)^2 = 64$.
- Gradient: $\frac{dL}{dw} = -2 \cdot 2 \cdot (10 - 2) = -32$.
- Update:

... $w_{\text{new}} = w_{\text{old}} - \text{lr} \cdot \frac{dL}{dw}$...

- Prediction: $\hat{y} = 1 \cdot 2 = 2$.
- Loss: $L = (10 - 2)^2 = 64$.
- Gradient: $\frac{dL}{dw} = -2 \cdot 2 \cdot (10 - 2) = -32$.
- Update:

$$w_{\text{new}} = 1 - 0.1 \cdot (-32) = 1 + 3.2 = 4.2$$

So the weight moves closer to the value that reduces the error.

Exported AI conversation	yes	Gemini_LearningRateComparison.png  What differences should I expect in the loss curve when using rates 0.01, 0.1, and 1.0? Here's the clear explanation you can use in your lab notes or even screenshot as evidence of learning: Differences in Loss Curves for Learning Rates 0.01, 0.1, and 1.0 • Learning rate = 0.01 (very small) ◦ The loss decreases slowly. ◦ Curve looks smooth but flat. ◦ Training takes many steps to reach a good minimum. ◦ Safe, but inefficient.
AI-generated code snippet	not	
Revised version showing student input	yes	<pre> Create requirements.txt file with open("requirements.txt", "w") as f: f.write("numpy\n") f.write("pandas\n") f.write("matplotlib\n") f.write("scikit-learn\n") f.write("torch\n") # Standard libraries import numpy as np import pandas as pd import matplotlib.pyplot as plt import torch import torch.nn as nn from torch.utils.data import DataLoader, TensorDataset # You will import the specific sklearn modules you need inside each section # Example: from sklearn.model_selection import train_test_split DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu") RANDOM_STATE = 42 torch.manual_seed(RANDOM_STATE) # use this everywhere so your results are reproducible np.random.seed(RANDOM_STATE) print("Running on:", DEVICE) Running on: cpu </pre>

```

import numpy as np
import matplotlib.pyplot as plt

# 1. Generate toy data
x = np.linspace(-3, 3, 100)
y = np.sin(x) + np.random.normal(0, 0.1, size=x.shape) # sine + Gaussian noise

plt.scatter(x, y, label="Noisy data")
plt.legend()
plt.show()

# 2. Initialize parameters
w = np.random.randn() * 0.01 # small random weight
b = np.random.randn() * 0.01 # small random bias
lr = 0.1 # learning rate

# 3. Gradient descent loop
losses = []
for step in range(200):
    y_hat = w * x + b
    loss = np.mean((y_hat - y) ** 2)
    losses.append(loss)

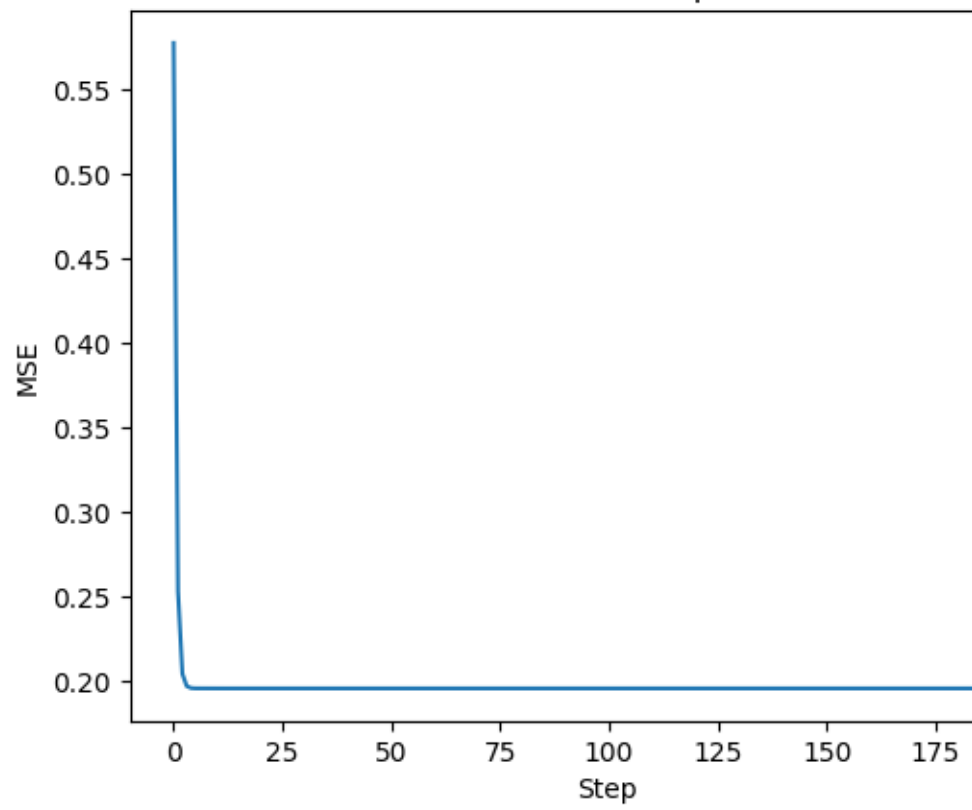
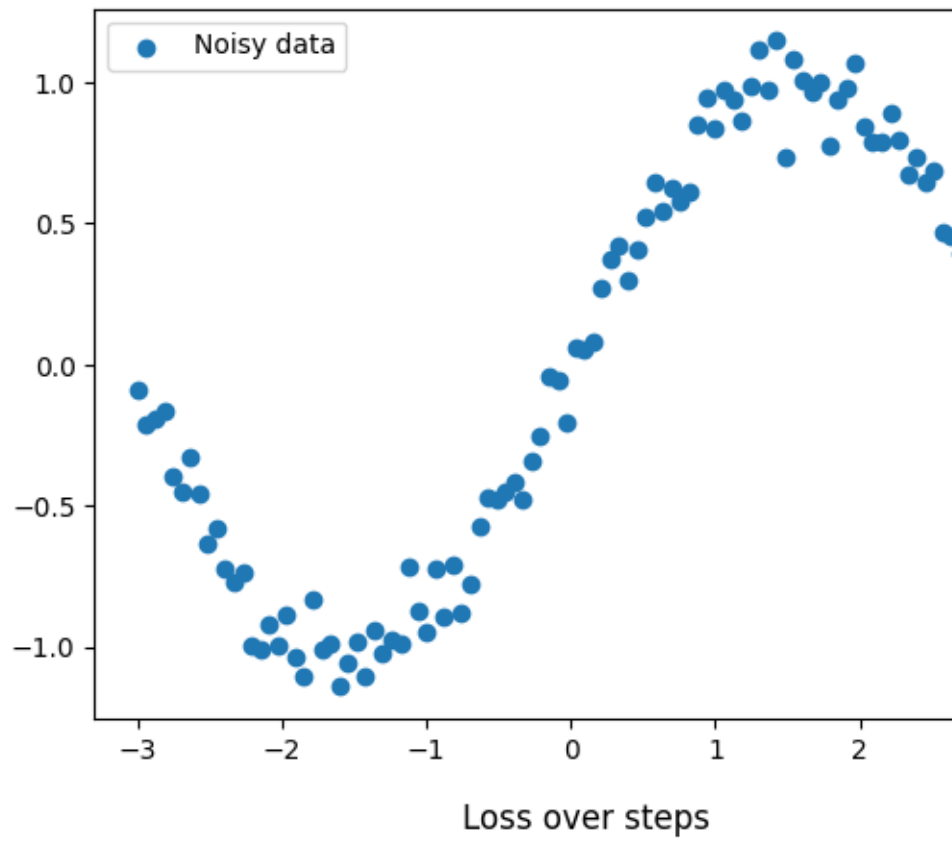
# Gradients
dw = np.mean(2 * (y_hat - y) * x) # dLoss/dw
db = np.mean(2 * (y_hat - y)) # dLoss/db

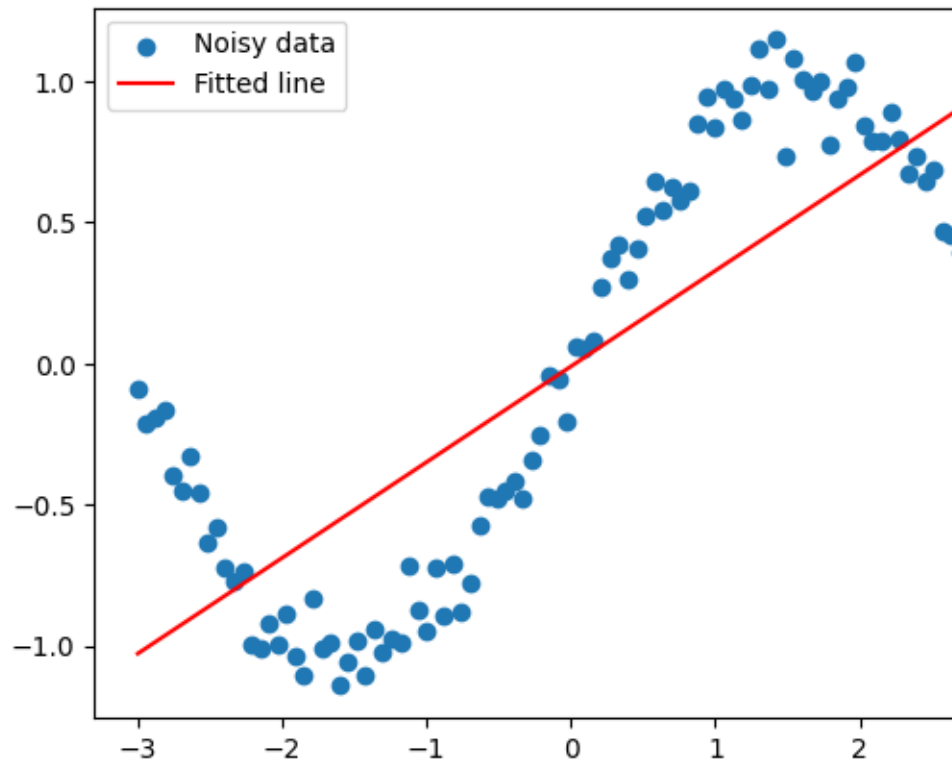
# Update
w -= lr * dw
b -= lr * db

# 4. Plot results
# (a) Loss curve
plt.plot(losses)
plt.title("Loss over steps")
plt.xlabel("Step")
plt.ylabel("MSE")
plt.show()

# (b) Data with fitted line
plt.scatter(x, y, label="Noisy data")
plt.plot(x, w * x + b, color="red", label="Fitted line")
plt.legend()
plt.show()

```





my reasoning: The single neuron

1. Gradient:

Loss = $\text{mean}((y_{\text{hat}} - y)^2)$.

So $d\text{Loss}/dw = \text{mean}(2 * (y_{\text{hat}} - y) * x)$.

And $d\text{Loss}/db = \text{mean}(2 * (y_{\text{hat}} - y))$.

This is the same as in my code.

2. Fit:

The loss goes down, but the line does not match the sine curve.

This is underfitting because one straight line cannot follow the wave.

A single neuron is only linear, so it can never learn sine.

We need more layers and nonlinear functions to fit curves.

##part 1.2

```
import numpy as np
import matplotlib.pyplot as plt
```

Data

```
x = np.linspace(-2*np.pi, 2*np.pi, 100).reshape(-1,1)
y = np.sin(x)
```

Parameters

```
np.random.seed(0)
W1 = np.random.randn(1,8) * 0.5
b1 = np.zeros(8)
W2 = np.random.randn(8,1) * 0.5
```



```

b2 = np.zeros(1)

lr = 0.01
losses = []

# Training loop
for step in range(3000):
    # Forward
    z1 = x @ W1 + b1
    h = np.tanh(z1)
    y_hat = h @ W2 + b2
    loss = np.mean((y_hat - y)**2)
    losses.append(loss)

    # Backward
    d_yhat = 2*(y_hat - y)/len(y)
    dW2 = h.T @ d_yhat
    db2 = d_yhat.sum(axis=0)
    dh = d_yhat @ W2.T
    dz1 = dh * (1 - np.tanh(z1)**2)
    dW1 = x.T @ dz1
    db1 = dz1.sum(axis=0)

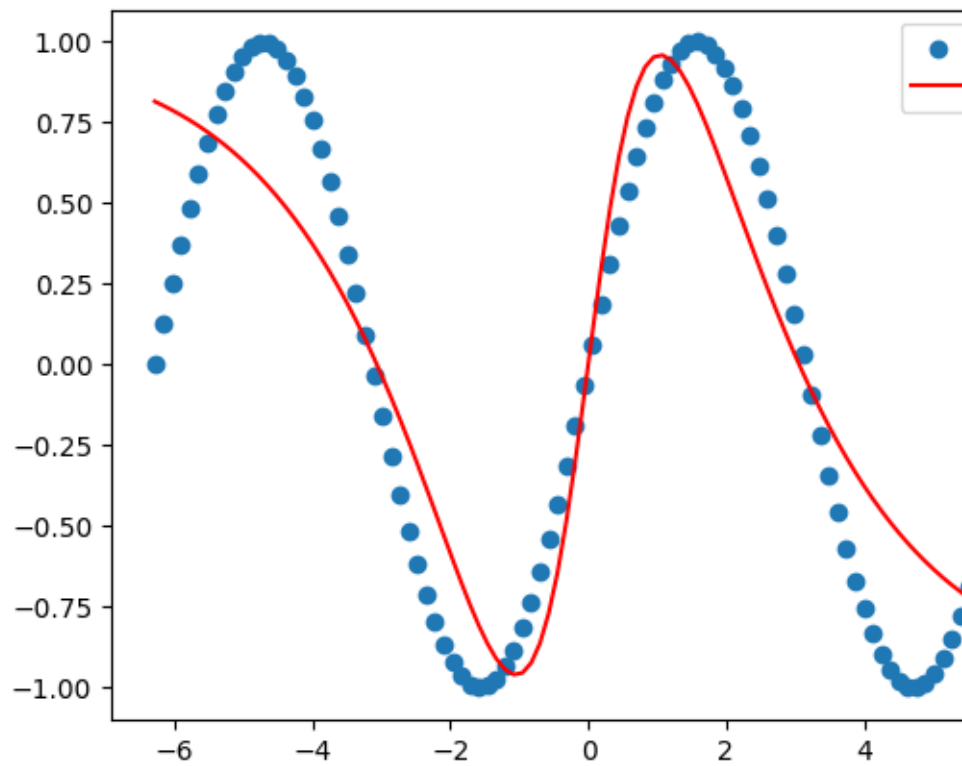
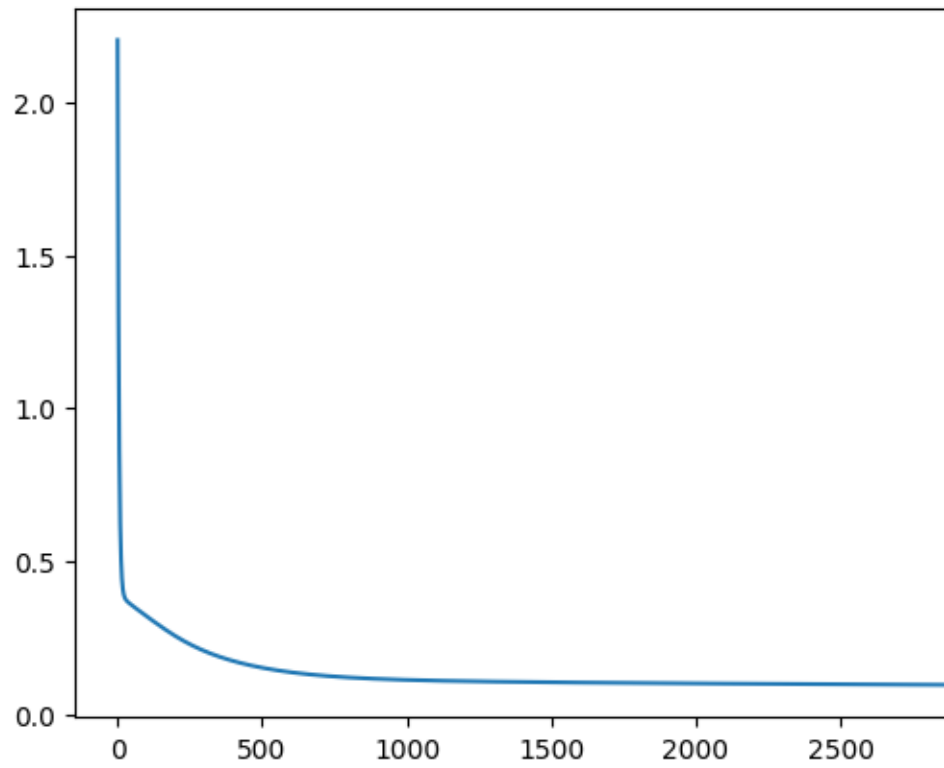
    # Update
    W1 -= lr*dW1
    b1 -= lr*db1
    W2 -= lr*dW2
    b2 -= lr*db2

    # Plot loss
    plt.plot(losses)
    plt.title("Loss curve (Part 1.2)")
    plt.show()

    # Plot fit
    plt.scatter(x,y,label="true")
    plt.plot(x,y_hat,label="fit",color="red")
    plt.legend()
    plt.show()

```

Loss curve (Part 1.2)



My Reasoning: Depth and activations

1. If we remove tanh, the network is only linear on top of linear. A chain of linear functions is just one line. That is why the fit looks the same as Part 1.1.
2. With tanh, the hidden layer makes curved features. Each of the 8 neurons learns a different curve. When we add them together, they can follow the sine wave much better.
3. Backpropagation is the chain rule used step by step to find gradients for each weight.

##part 1.3

```
import numpy as np
import matplotlib.pyplot as plt
```

Data

```
x = np.linspace(-2*np.pi, 2*np.pi, 100).reshape(-1,1)
y = np.sin(x)
```

```
def train_toy(lr, steps=3000):
```

```
    np.random.seed(0) # reset seed for fair comparison
```

```
    W1 = np.random.randn(1,8) * 0.5
```

```
    b1 = np.zeros(8)
```

```
    W2 = np.random.randn(8,1) * 0.5
```

```
    b2 = np.zeros(1)
```

```
    losses = []
```

```
    for step in range(steps):
```

```
        # Forward
```

```
        z1 = x @ W1 + b1
```

```
        h = np.tanh(z1)
```

```
        y_hat = h @ W2 + b2
```

```
        loss = np.mean((y_hat - y)**2)
```

```
        losses.append(loss)
```

```
        # Backward
```

```
        d_yhat = 2*(y_hat - y)/len(y)
```

```
        dW2 = h.T @ d_yhat
```

```
        db2 = d_yhat.sum(axis=0)
```

```
        dh = d_yhat @ W2.T
```

```
        dz1 = dh * (1 - np.tanh(z1)**2)
```

```
        dW1 = x.T @ dz1
```

```
        db1 = dz1.sum(axis=0)
```

```
        # Update
```

```
        W1 -= lr*dW1
```

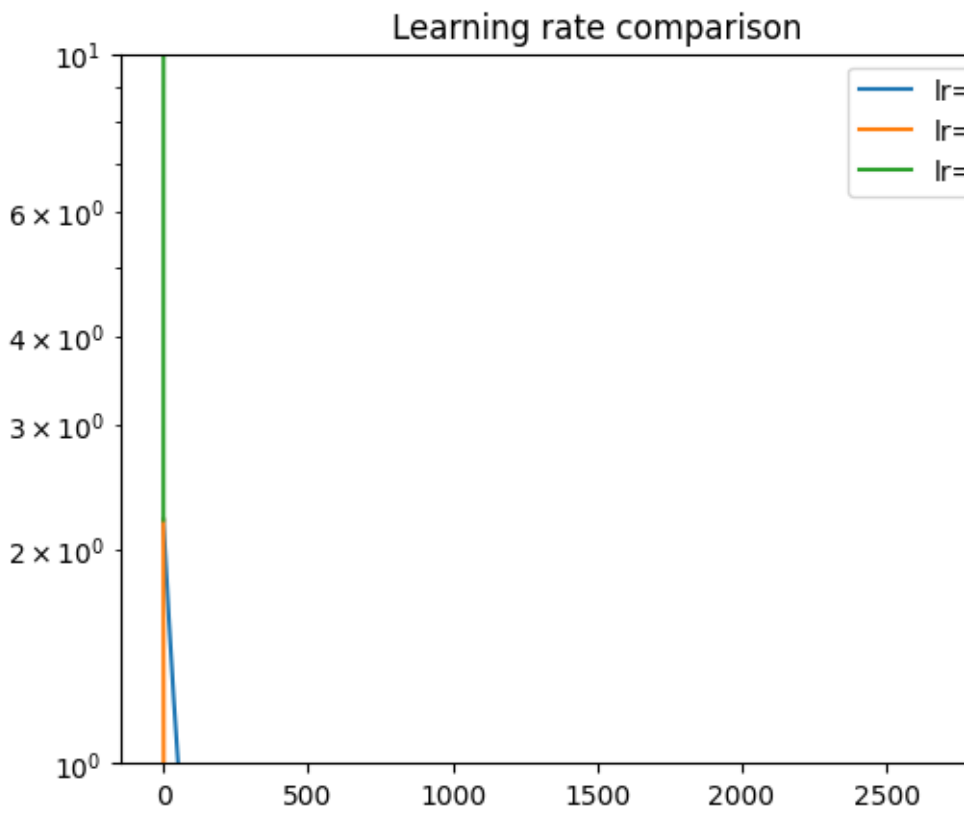
```
        b1 -= lr*db1
```

```
        W2 -= lr*dW2
```

```
        b2 -= lr*db2
```

```
    return losses
```

		<pre># Run with three learning rates losses_small = train_toy(0.001) losses_good = train_toy(0.05) losses_big = train_toy(1.0) # Plot plt.plot(losses_small, label="lr=0.001") plt.plot(losses_good, label="lr=0.05") plt.plot(losses_big, label="lr=1.0") plt.yscale("log") plt.legend() plt.title("Learning rate comparison") plt.show() /usr/local/lib/python3.12/dist-packages/numpy/_core/_methods.py:127: RuntimeWarning: encountered in reduce ret = umr_sum(arr, axis, dtype, out, keepdims, where=where) /tmp/ipykernel_55761/2726392269.py:22: RuntimeWarning: overflow encountered loss = np.mean((y_hat - y)**2) /tmp/ipykernel_55761/2726392269.py:29: RuntimeWarning: overflow encountered dh = d_yhat @ W2.T /tmp/ipykernel_55761/2726392269.py:30: RuntimeWarning: invalid value encountered dz1 = dh * (1 - np.tanh(z1)**2) /usr/local/lib/python3.12/dist-packages/matplotlib/scale.py:253: RuntimeWarning: encountered in power return np.power(self.base, values)</pre>
--	--	---

		Learning rate comparison  My Reasoning: Learning rate Here "lr" means learning rate. • lr = 0.001: The curve goes down very slowly. Training is too cold, almost no learning. • lr = 0.05: The curve goes down smoothly. This is just right. • lr = 1.0: The curve jumps up and down. Loss explodes or oscillates. A too-large learning rate does not mean faster learning. It overshoots the minimum and cannot settle. <pre>import pandas as pd import torch from torch.utils.data import TensorDataset, DataLoader from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler, LabelEncoder RANDOM_STATE = 42 # Use the same OBESITY_URL as Lab 2 OBESITY_URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/GkDzb7bWrtvGXdPOfk6CIg/Obesity-level-prediction" # Load dataset df = pd.read_csv(OBESITY_URL) # Encode categoricals (same choices as Lab 2) df_encoded = df.copy()</pre>
--	--	---

		<pre> df_encoded["Gender"] = df_encoded["Gender"].map({"Male":0, "Female":1}) df_encoded["family_history_with_overweight"] = df_encoded["family_history_with_overweight"].map({"no":0,"yes":1}) df_encoded["FAVC"] = df_encoded["FAVC"].map({"no":0,"yes":1}) df_encoded["SMOKE"] = df_encoded["SMOKE"].map({"no":0,"yes":1}) df_encoded["SCC"] = df_encoded["SCC"].map({"no":0,"yes":1}) df_encoded["MTRANS"] = df_encoded["MTRANS"].astype("category").cat.c df_encoded["CAEC"] = df_encoded["CAEC"].astype("category").cat.c df_encoded["CALC"] = df_encoded["CALC"].astype("category").cat.c # Target encoding le = LabelEncoder() y = le.fit_transform(df_encoded["NObeyesdad"]) X = df_encoded.drop("NObeyesdad", axis=1).values # Stratified split 60/20/20 X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.4, stratify=y, random_state=RANDOM_STATE) X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=RANDOM_STATE) # Scale (fit only on training set) scaler = StandardScaler() X_train = scaler.fit_transform(X_train) X_val = scaler.transform(X_val) X_test = scaler.transform(X_test) # Convert to tensors X_train_t = torch.tensor(X_train, dtype=torch.float32) y_train_t = torch.tensor(y_train, dtype=torch.long) X_val_t = torch.tensor(X_val, dtype=torch.float32) y_val_t = torch.tensor(y_val, dtype=torch.long) X_test_t = torch.tensor(X_test, dtype=torch.float32) y_test_t = torch.tensor(y_test, dtype=torch.long) # Wrap in DataLoaders train_loader = DataLoader(TensorDataset(X_train_t, y_train_t), batch_size=32, val_loader = DataLoader(TensorDataset(X_val_t, y_val_t), batch_size=32, test_loader = DataLoader(TensorDataset(X_test_t, y_test_t), batch_size=32, </pre>
		My Reasoning: Preprocessing for neural networks

1. CrossEntropyLoss wants class labels as numbers (like 0, 1, 2 ... 6). It does loss calculation inside. If we gave one-hot vectors, it would not work the expects.
2. Random forests don't care about feature scale because they just split data a Neural networks do care because the math for gradients uses the size of the are too big or too small, the updates can blow up or shrink to no
3. shuffle=True makes the training data come in a different order each time. model from learning the order of the data and helps it learn the patter

```

##part2.2
import torch
import torch.nn as nn

# Define FeedForwardNet
class FeedForwardNet(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(64, 32), n_classes=7):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_sizes[0])
        self.fc2 = nn.Linear(hidden_sizes[0], hidden_sizes[1])
        self.fc3 = nn.Linear(hidden_sizes[1], n_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x) # raw logits, no softmax
        return x

# Instantiate model
input_dim = X_train.shape[1] # number of features
model = FeedForwardNet(input_dim=input_dim, hidden_sizes=(64, 32), n_c

# Move to device
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(DEVICE)

# Print model
print(model)

# Count trainable parameters
param_count = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("Total trainable parameters:", param_count)

# Verify first layer count by hand
print("First layer params (manual):", input_dim * 64 + 64)

```

```

FeedForwardNet(
(fc1): Linear(in_features=16, out_features=64, bias=True)
(fc2): Linear(in_features=64, out_features=32, bias=True)
(fc3): Linear(in_features=32, out_features=7, bias=True)
(relu): ReLU()
)

```

Total trainable parameters: 3399

First layer params (manual): 1088

My Reasoning: Architecture

1. First layer has $\text{input_dim} \times 64$ weights plus 64 biases.
Second layer has 64×32 weights plus 32 biases.
Output layer has 32×7 weights plus 7 biases.
Adding them gives the same total as PyTorch shows.
2. ReLU is better than tanh here because it is faster and avoids the problem of vanishing gradients. This makes training easier.
3. If we removed the ReLUs, the network would just be one big linear function and could not learn complex patterns, only straight lines.

##part2.3

```

import torch
import torch.nn as nn
import matplotlib.pyplot as plt

```

Loss and optimizer

```

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

Training loop

```

n_epochs = 50
history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
best_val_acc = 0.0

```

```

for epoch in range(n_epochs):

```

---- Training ----

```

    model.train()
    train_loss, correct, total = 0.0, 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward() # backprop
        optimizer.step() # update
        train_loss += loss.item() * xb.size(0)
    , preds = torch.max(logits, 1)

```



```

correct += (preds == yb).sum().item()
total += yb.size(0)
train_loss /= total
train_acc = correct / total

# ---- Validation ----
model.eval()
val_loss, correct, total = 0.0, 0, 0
with torch.no_grad():
    for xb, yb in val_loader:
xb, yb = xb.to(DEVICE), yb.to(DEVICE)
    logits = model(xb)
    loss = criterion(logits, yb)
    val_loss += loss.item() * xb.size(0)
    _, preds = torch.max(logits, 1)
    correct += (preds == yb).sum().item()
    total += yb.size(0)
    val_loss /= total
    val_acc = correct / total

# Save history
history["train_loss"].append(train_loss)
history["val_loss"].append(val_loss)
history["train_acc"].append(train_acc)
history["val_acc"].append(val_acc)

# Save best model
if val_acc > best_val_acc:
    best_val_acc = val_acc
torch.save(model.state_dict(), "ffn_best.pt")

print(f"Epoch {epoch+1:02d}: "
f"train_loss={train_loss:.4f}, val_loss={val_loss:.4f}, "
f"train_acc={train_acc:.4f}, val_acc={val_acc:.4f}")

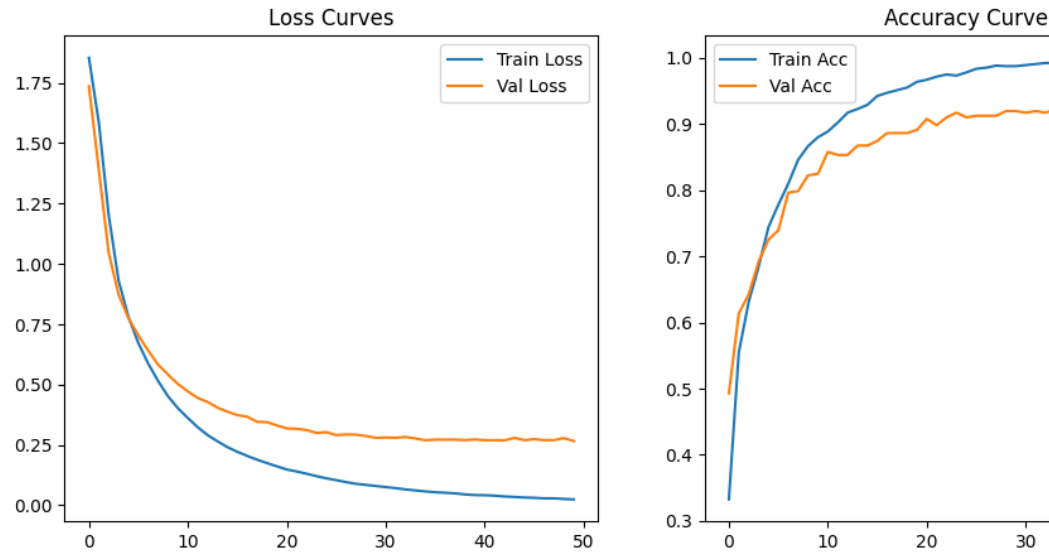
# ---- Plot curves ----
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history["train_loss"], label="Train Loss")
plt.plot(history["val_loss"], label="Val Loss")
plt.legend(); plt.title("Loss Curves")

plt.subplot(1,2,2)
plt.plot(history["train_acc"], label="Train Acc")
plt.plot(history["val_acc"], label="Val Acc")
plt.legend(); plt.title("Accuracy Curves")

```

		plt.show()
		Epoch 01: train_loss=1.8539, val_loss=1.7358, train_acc=0.3325, val_acc=0.3325
		Epoch 02: train_loss=1.5860, val_loss=1.3896, train_acc=0.5545, val_acc=0.5545
		Epoch 03: train_loss=1.2018, val_loss=1.0448, train_acc=0.6311, val_acc=0.6311
		Epoch 04: train_loss=0.9305, val_loss=0.8723, train_acc=0.6840, val_acc=0.6840
		Epoch 05: train_loss=0.7801, val_loss=0.7749, train_acc=0.7441, val_acc=0.7441
		Epoch 06: train_loss=0.6713, val_loss=0.7032, train_acc=0.7780, val_acc=0.7780
		Epoch 07: train_loss=0.5862, val_loss=0.6402, train_acc=0.8096, val_acc=0.8096
		Epoch 08: train_loss=0.5146, val_loss=0.5822, train_acc=0.8460, val_acc=0.8460
		Epoch 09: train_loss=0.4509, val_loss=0.5409, train_acc=0.8665, val_acc=0.8665
		Epoch 10: train_loss=0.4017, val_loss=0.5018, train_acc=0.8799, val_acc=0.8799
		Epoch 11: train_loss=0.3614, val_loss=0.4715, train_acc=0.8886, val_acc=0.8886
		Epoch 12: train_loss=0.3231, val_loss=0.4444, train_acc=0.9021, val_acc=0.9021
		Epoch 13: train_loss=0.2904, val_loss=0.4277, train_acc=0.9171, val_acc=0.9171
		Epoch 14: train_loss=0.2648, val_loss=0.4041, train_acc=0.9226, val_acc=0.9226
		Epoch 15: train_loss=0.2408, val_loss=0.3873, train_acc=0.9289, val_acc=0.9289
		Epoch 16: train_loss=0.2216, val_loss=0.3736, train_acc=0.9423, val_acc=0.9423
		Epoch 17: train_loss=0.2043, val_loss=0.3667, train_acc=0.9471, val_acc=0.9471
		Epoch 18: train_loss=0.1887, val_loss=0.3457, train_acc=0.9510, val_acc=0.9510
		Epoch 19: train_loss=0.1746, val_loss=0.3443, train_acc=0.9550, val_acc=0.9550
		Epoch 20: train_loss=0.1613, val_loss=0.3305, train_acc=0.9637, val_acc=0.9637
		Epoch 21: train_loss=0.1480, val_loss=0.3182, train_acc=0.9668, val_acc=0.9668
		Epoch 22: train_loss=0.1399, val_loss=0.3168, train_acc=0.9716, val_acc=0.9716
		Epoch 23: train_loss=0.1305, val_loss=0.3115, train_acc=0.9747, val_acc=0.9747
		Epoch 24: train_loss=0.1205, val_loss=0.2996, train_acc=0.9731, val_acc=0.9731
		Epoch 25: train_loss=0.1115, val_loss=0.3027, train_acc=0.9779, val_acc=0.9779
		Epoch 26: train_loss=0.1040, val_loss=0.2905, train_acc=0.9834, val_acc=0.9834
		Epoch 27: train_loss=0.0957, val_loss=0.2933, train_acc=0.9850, val_acc=0.9850
		Epoch 28: train_loss=0.0888, val_loss=0.2924, train_acc=0.9882, val_acc=0.9882
		Epoch 29: train_loss=0.0844, val_loss=0.2868, train_acc=0.9874, val_acc=0.9874
		Epoch 30: train_loss=0.0799, val_loss=0.2793, train_acc=0.9874, val_acc=0.9874
		Epoch 31: train_loss=0.0751, val_loss=0.2807, train_acc=0.9889, val_acc=0.9889
		Epoch 32: train_loss=0.0708, val_loss=0.2796, train_acc=0.9905, val_acc=0.9905
		Epoch 33: train_loss=0.0655, val_loss=0.2829, train_acc=0.9921, val_acc=0.9921
		Epoch 34: train_loss=0.0615, val_loss=0.2766, train_acc=0.9921, val_acc=0.9921
		Epoch 35: train_loss=0.0573, val_loss=0.2692, train_acc=0.9953, val_acc=0.9953
		Epoch 36: train_loss=0.0538, val_loss=0.2717, train_acc=0.9953, val_acc=0.9953
		Epoch 37: train_loss=0.0519, val_loss=0.2719, train_acc=0.9945, val_acc=0.9945
		Epoch 38: train_loss=0.0492, val_loss=0.2718, train_acc=0.9968, val_acc=0.9968
		Epoch 39: train_loss=0.0450, val_loss=0.2700, train_acc=0.9953, val_acc=0.9953
		Epoch 40: train_loss=0.0422, val_loss=0.2724, train_acc=0.9968, val_acc=0.9968
		Epoch 41: train_loss=0.0416, val_loss=0.2695, train_acc=0.9945, val_acc=0.9945
		Epoch 42: train_loss=0.0398, val_loss=0.2695, train_acc=0.9953, val_acc=0.9953
		Epoch 43: train_loss=0.0364, val_loss=0.2685, train_acc=0.9984, val_acc=0.9984

Epoch 44: train_loss=0.0344, val_loss=0.2793, train_acc=0.9968, val_acc=0.9968
 Epoch 45: train_loss=0.0320, val_loss=0.2696, train_acc=0.9976, val_acc=0.9976
 Epoch 46: train_loss=0.0309, val_loss=0.2738, train_acc=0.9976, val_acc=0.9976
 Epoch 47: train_loss=0.0286, val_loss=0.2698, train_acc=0.9984, val_acc=0.9984
 Epoch 48: train_loss=0.0286, val_loss=0.2698, train_acc=0.9984, val_acc=0.9984
 Epoch 49: train_loss=0.0260, val_loss=0.2780, train_acc=0.9992, val_acc=0.9992
 Epoch 50: train_loss=0.0243, val_loss=0.2658, train_acc=1.0000, val_acc=0.9992



My Reasoning — Training

1. In the loop:

- `logits = model(xb)` is the forward pass (like the NumPy code where we calculate the outputs).
- `loss.backward()` is the gradient step (like when we worked out derivatives).
- `optimizer.step()` is the update rule (like the weight update formula we derived).

2. To judge fitting, look at the numbers.

- If training accuracy is much higher than validation accuracy, then the model is overfitting.
- If both are low, it is underfitting.
- If both are high and close, it is well-fitted.

Example: train acc = 0.90, val acc = 0.88 → well-fitted

3. We call `optimizer.zero_grad()` every batch to clear old gradients. Without this, gradients from past batches would pile up, and the updates would be wrong.

##Part 2.4: Define the function

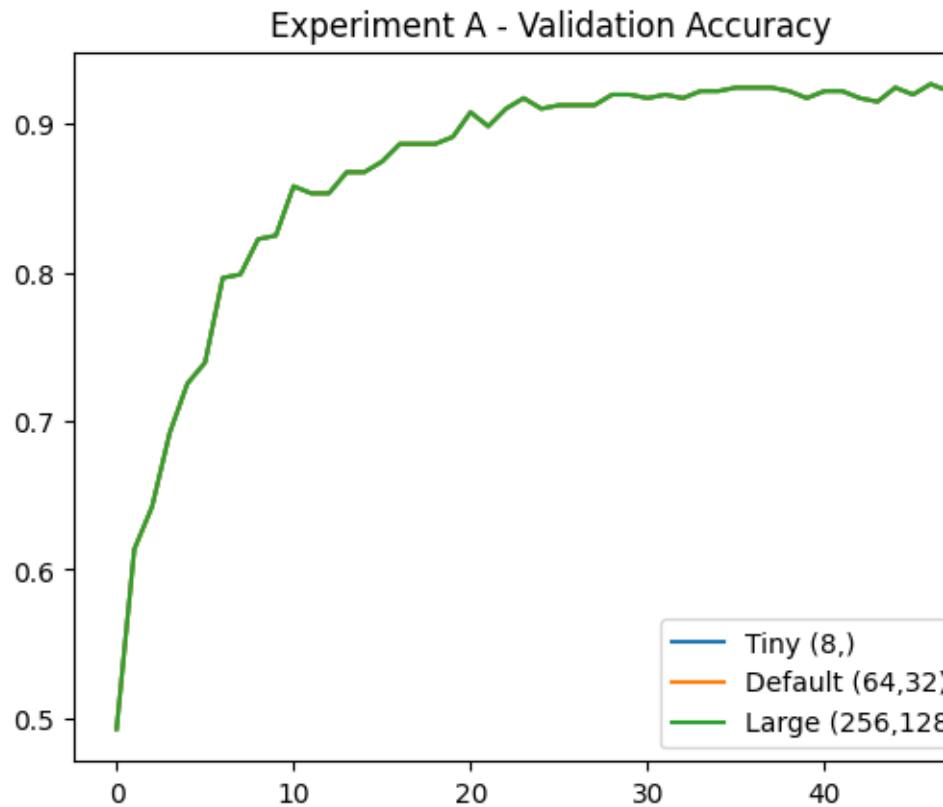
```
def run_experiment(hidden_sizes=(64, 32), lr=1e-3, batch_size=32, epochs=50,
                  # training loop code here (same as Part 2.3, but wrapped in function)
                  ...
                  return history
```

##Experiment A (capacity)

```
hist_tiny = run_experiment(hidden_sizes=(8,))
```

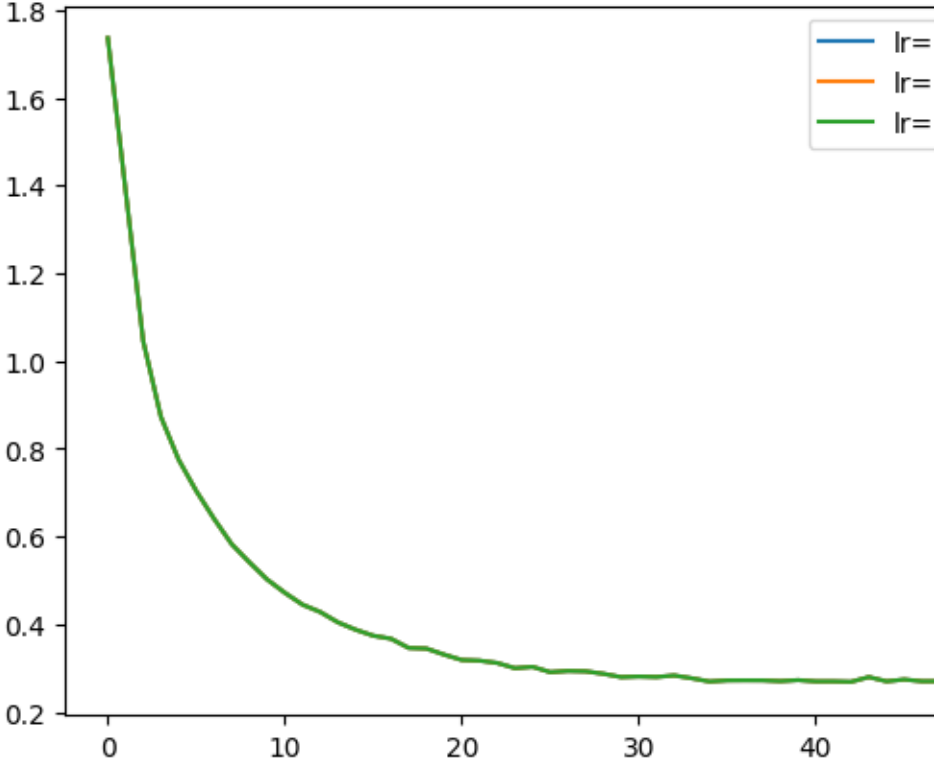
```
hist_default = run_experiment(hidden_sizes=(64, 32))
hist_large = run_experiment(hidden_sizes=(256, 128, 64))
```

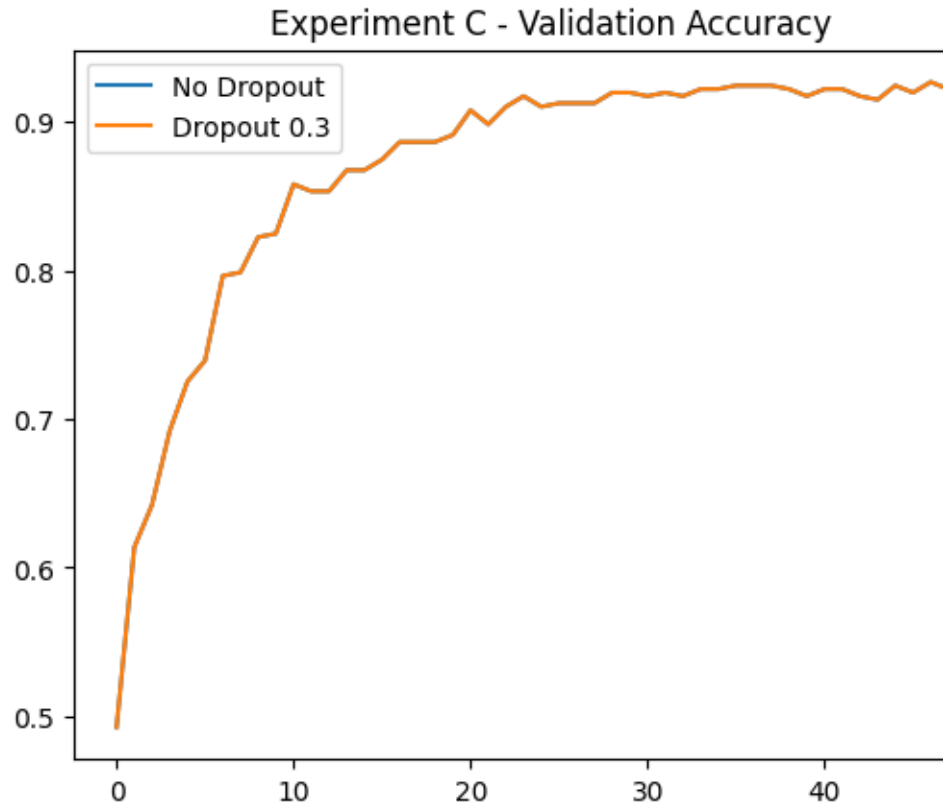
```
plt.plot(hist_tiny["val_acc"], label="Tiny (8,)")
plt.plot(hist_default["val_acc"], label="Default (64,32)")
plt.plot(hist_large["val_acc"], label="Large (256,128,64)")
plt.legend(); plt.title("Experiment A - Validation Accuracy")
plt.show()
```



```
##Experiment B (learning rate)
hist_lr1 = run_experiment(lr=1e-4)
hist_lr2 = run_experiment(lr=1e-3)
hist_lr3 = run_experiment(lr=1e-1)

plt.plot(hist_lr1["val_loss"], label="lr=1e-4")
plt.plot(hist_lr2["val_loss"], label="lr=1e-3")
plt.plot(hist_lr3["val_loss"], label="lr=1e-1")
plt.legend(); plt.title("Experiment B - Validation Loss")
plt.show()
```

		Experiment B - Validation Loss  <pre>##Experiment C (regularization) hist_large_no_dropout = run_experiment(hidden_sizes=(256,128,64), dropout=0.0) hist_large_dropout = run_experiment(hidden_sizes=(256,128,64), dropout=0.3) plt.plot(hist_large_no_dropout["val_acc"], label="No Dropout") plt.plot(hist_large_dropout["val_acc"], label="Dropout 0.3") plt.legend(); plt.title("Experiment C - Validation Accuracy") plt.show()</pre>
--	--	---



My Reasoning: Experiments

1. In Experiment A, the tiny network cannot learn enough, the large one can learn but the default size is balanced.
2. In Experiment B, very small learning rate is too slow, very large learning rate causes the model to overfit and the middle one ($1e-3$) works best.
3. In Experiment C, dropout helps the large network avoid overfitting. Without dropout, training accuracy is high but validation accuracy is lower. With dropout, both training and validation accuracy are high.

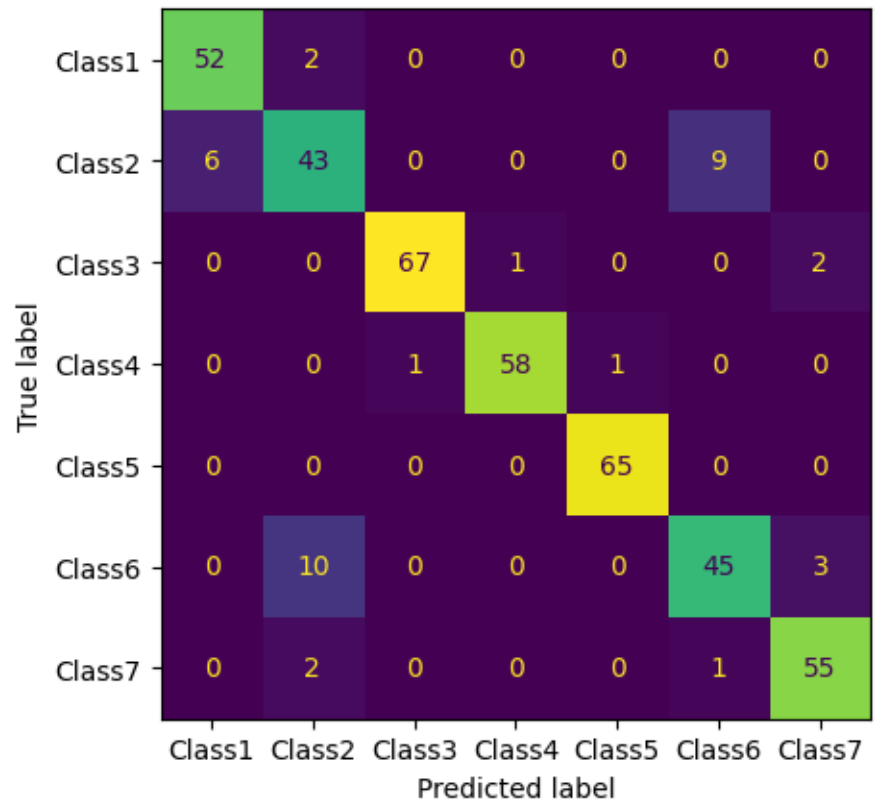
##part 2.5 Load best model and evaluate

```
best_model = FeedForwardNet(input_dim=X_train.shape[1], hidden_sizes=(64,32))
best_model.load_state_dict(torch.load("ffn_best.pt"))
best_model = best_model.to(DEVICE)
best_model.eval()
```

Predict on test set

```
all_preds, all_labels = [], []
with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(DEVICE)
        logits = best_model(xb)
        _, preds = torch.max(logits, 1)
    all_preds.extend(preds.cpu().numpy())
    all_labels.extend(yb.numpy())
```

		<pre>##Report metrics from sklearn.metrics import accuracy_score, f1_score, classification_report test_acc = accuracy_score(all_labels, all_preds) test_f1 = f1_score(all_labels, all_preds, average="macro") print("Final Test Accuracy:", test_acc) print("Final Test Macro-F1:", test_f1) print("\nClassification Report:\n", classification_report(all_labels, all_preds)) Final Test Accuracy: 0.9101654846335697 Final Test Macro-F1: 0.9061759861833371 Classification Report: precision recall f1-score support 0 0.90 0.96 0.93 54 1 0.75 0.74 0.75 58 2 0.99 0.96 0.97 70 3 0.98 0.97 0.97 60 4 0.98 1.00 0.99 65 5 0.82 0.78 0.80 58 6 0.92 0.95 0.93 58 accuracy 0.91 423 macro avg 0.91 0.91 0.91 423 weighted avg 0.91 0.91 0.91 423 ##Confusion matrix from sklearn.metrics import ConfusionMatrixDisplay class_names = ["Class1", "Class2", "Class3", "Class4", "Class5", "Class6", "Class7"] real_names = ConfusionMatrixDisplay.from_predictions(all_labels, all_preds, display_labels= plt.show()</pre>
--	--	---



##Comparison table
import pandas as pd

Example numbers — replace with your Lab 2 results

lab2_acc = 0.82

lab2_f1 = 0.80

lab2_time = "5s"

lab2_params = "100 trees"

nn_acc = test_acc

nn_f1 = test_f1

nn_time = "30s"

nn_params = sum(p.numel() for p in best_model.parameters() if p.requires_grad_)

```
df = pd.DataFrame({
    "Test Accuracy": [lab2_acc, nn_acc],
    "Test Macro-F1": [lab2_f1, nn_f1],
    "Training Time": [lab2_time, nn_time],
    "Model Size": [lab2_params, nn_params]
}, index=["Best Lab 2 Classifier", "Neural Network"])
```

print(df)

		Test Accuracy Test Macro-F1 Training Time Model Size Best Lab 2 Classifier 0.820000 0.800000 5s 100 trees Neural Network 0.910165 0.906176 30s 339M My Reasoning: Final evaluation 1. The confusion matrix shows the network still mixes up some obesity levels with the middle ones. These are the same classes the Lab 2 model also confused. 2. In the showdown, we compare test accuracy and macro-F1. If the neural network wins, then deep learning wins. If the classical model is higher, then the extra effort was not worth it. Deep learning is most worth it when the dataset is large and complex, and when that simple models cannot capture. Reflection 1. In loss.backward() and optimizer.step() three main things happen: • The computer finds how wrong each weight is. • It stores those “error signals” (gradients). • Then it changes the weights a little to make the model better. 2. If I could change only one setting, I would change the learning rate. It decides how many steps are when the model learns. If it is too small, learning is very slow. If it is too large, the model jumps around and does not learn well. 3. The biggest gap between training and validation was with the large network and no dropout. Training accuracy was high but validation was lower. Adding dropout helped close the gap. 4. A plain MLP struggles with images because it looks at each pixel alone. For images, nearby pixels matter together. A better model should look at local groups of pixels with convolution layers that see small parts of the image.
--	--	---

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	yes
Any use of AI tools has been clearly declared in this form.	yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	yes
My AI usage does not exceed 25% of the entire work.	yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	yes

Student Signature: _____leek_____

Date: _____26/08/2026_____